KEMENTERIAN LINGKUNGAN HIDUP /
BADAN PENGENDALIAN LINGKUNGAN HIDUP
DEPUTI BIDANG PENGENDALIAN PENCEMARAN DAN KERUSAKAN LINGKUNGAN
DIREKTORAT PERLINDUNGAN DAN PENGELOLAAN MUTU AIR

TERMS AND PROCEDURES FOR CONTINUOUS WASTE WATER MONITORING DATA SENDING (SPARING) IN 2025

A. INTRODUCTION

1. URL : **sparing.kemenlh.go.id**
2. Monitoring Data Delivery is per hour, data with an interval of 2 minutes, so the total data sent is 30 (thirty) data, in one data delivery package
3. The data sent at the Nth hour is the data at the N-1st hour, for example, at 09.00, it will send data at 08.00.00, 08.02.00, 08.04.00, 08.06.00, …, 08.58.00

B. (REQUEST) API SECRET

1. API SECRET applies generally (independent of the Company) and can change at any time, without notice.
2. Functions as a SECRET KEY for the data encryption process
3. Implementation :
   i. API URL : https://sparing.kemenlh.go.id/api/secret-sensor
   ii. METHOD : GET

C. ID LOGGER

1. LOGGER ID is a unique code for each LOGGER owned by the company and has been registered by KLH/BPLH
2. LOGGER ID is used as UID (Unique Identification) when sending data

D. DATA SENDING FORMAT

1. DATA VARIABLES:

i. uid: STRING data type in the form of a LOGGER ID;
ii. datetime: INTEGER data type in the form of a time in UNIX TIMESTAMP format
iii. pH: NUMBER data type in the form of a pH concentration value
iv. cod: NUMBER data type in the form of a COD concentration value (mg/l);
v. tss: NUMBER data type in the form of a TSS concentration value (mg/l);
vi. nh3n: NUMBER data type in the form of a NH3N concentration value (mg/l);
vii. debit: NUMBER data type in the form of a DEBIT concentration value (m3/minute);

2. DATA TYPE:

i. STRING is a data type consisting of numbers and/or alphabets.
ii. INTEGER is a data type consisting of whole numbers;
iii. NUMBER is a data type consisting of whole numbers or decimal numbers;
iv. UNIX TIMESTAMP is an INTEGER data type that represents time;

PHP	`time()` More PHP
Python	`import time; time.time()`
Ruby	`Time.now` (or `Time.new`). To display the epoch: `Time.now.to_i`
Perl	`time` More Perl
Java	`long epoch = System.currentTimeMillis()/1000;` Returns epoch in seconds.
C#	`DateTimeOffset.Now.ToUnixTimeSeconds()` (.NET Framework 4.6+/.NET Core), older versions: `var epoch = (DateTime.UtcNow - new DateTime(1970, 1, 1, 0, 0, 0, DateTimeKind.Utc)).TotalSeconds;`
Objective-C	`[[NSDate date] timeIntervalSince1970];` (returns double) or `NSString *currentTimestamp = [NSString stringWithFormat:@"%f", [[NSDate date] timeIntervalSince1970]];`
C++11	`double now = std::chrono::duration_cast<std::chrono::seconds> (std::chrono::system_clock::now().time_since_epoch()).count();`

3. **DATA FORMAT:**
   i. Data format is in JSON (JavaScript Object Notation);
   ii. JSON is a popular data format used in system/application integration;

4. **DATA DELIVERY PER 2 MINUTES;**

```
{
    "uid": "ABC3fe3fa2366e1acce66XYZ",
    "datetime": 1737972001,
    "pH": 6.20,
    "cod": 4.12,
    "tss": 250,
    "nh3n": 4.14,
    "debit": 3.12
}
```

5. **HOURLY DATA DELIVERY (2 MINUTE INTERVAL);**

```
{
    "uid": "ABC3fe3fa2366e1acce66XYZ",
    "data": [
        {
            "datetime": 1737968400,
            "pH": 1,
            "cod": 2,
            "tss": 3,
            "nh3n": 4,
            "debit": 5
        },

        …… JUMLAH DATA 30 ……

        {
            "datetime": 1737971880,
            "pH": 1,
            "cod": 2,
            "tss": 3,
            "nh3n": 4,
            "debit": 5
        }
    ]
}
```

E. DATA ENCRYPTION

1. Encryption using JWT (JSON Web Tokens), a reference can be found at https://jwt.io/;
2. JWT Header;

```
{
    "typ": "JWT",
    "alg": "HS256"
}
```

3. Encrypt the HEADER and DATA using Base64UrlEncode and HMACSHA256;
4. Combine the results of the HEADER encryption and DATA encryption with the dot (.) character;

```
dataEncode = Base64UrlEncode(HEADER) + "." + Base64UrlEncode(DATA);
```

5. Create SECRET KEY from API KEY & DATA ENCODE using HMACSHA256;
6. KEY SECRET encryption using Base64UrlEncode;

```
keySecret = Base64UrlEncode(HMACSHA256(dataEncode, API_KEY));
```

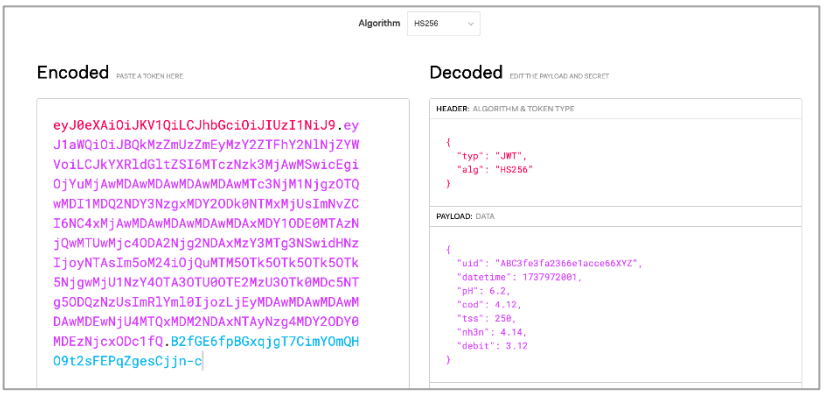
7. Combine dataEncode and keySecret with a Dot (.) Character;

```
dataSend = dataEncode + "." + keySecret;
```

8. SEND DATA in JWT format;


```
{
    "token":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1aWQiOiJBQkMzZmUzZmEyMzY2ZTF
hY2NlNjZYWVoiLCJkYXRldGltZSI6MTczNzk3MjAwMSwicEgiOjYuMjAwMDAwMDAwMDAw
MDAwMTc3NjM1NjgzOTQwMDI1MDQ2NDY3NzgxMDY2ODk0NTMxMjUsImNvZCI6NC4xMjAwM
DAwMDAwMDAwMDAxMDY1ODE0MTAzNjQwMTUwMjc4ODA2Njg2NDAxMzY3MTg3NSwidHNzIj
oyNTAsIm5oM24iOjQuMTM5OTk5OTk5OTk5OTk5NjgwMjU1NzY4OTA3OTU0OTE2MzU3OTk
0MDc5NTg5ODQzNzUsImRlYml0IjozLjEyMDAwMDAwMDAwMDAwMDEwNjU4MTQxMDM2NDAx
NTAyNzg4MDY2ODY0MDEzNjcxODc1fQ.B2fGE6fpBGxqjgT7CimYOmQHO9t2sFEPqZgesC
jjn-c"
}
```


9. To ensure that the data sent is correct, it can be checked via https://jwt.io/;



10. Examples of encryption usage:

i. The JWT Programming Language Library can be found at https://jwt.io/libraries;

ii. PHP Programming Language;

```
function base64url_encode($data){
            $b64 = base64_encode($data);
            if ($b64 === false) return false;
            $url = strtr($b64, '+/', '-_');
            return rtrim($url, '=');
}
$password = "API_SECRET";
$header = ["typ" => "JWT", "alg" => "HS256"];
$jsonHeader = json_encode($header);

$data2Minute = [
            "uid" => "ABC3fe3fa2366e1acce66XYZ",
            "datetime" => strtotime("2025-01-10 08:00:00"),
            "pH" => 6.2,
            "cod" => 4.12,
            "tss" => 250,
            "nh3n" => 4.14,
            "debit" => 3.12
      ];
$jsonData2Minute = json_encode($data2Minute);

$dataConversion = base64url_encode($jsonHeader) . '.' .
base64url_encode($jsonData2Minute);
$keyConversion = base64url_encode(hash_hmac('sha256', $dataConversion, $password,
true));
$dataKey = $dataConversion . '.' . $keyConversion;
$dataSend = ['token' => $dataKey];

$jsonDataSend = json_encode($dataSend);
echo $jsonDataSend;
```

iii. Python;

```
import jwt
jwtHeaders = {
        "alg" : "HS256",
        "typ" : "JWT"
}
jwtPayload = {
        "name" : "Test",
        "email" : "test@gmail.com",
        "admin" : "true",
        "exp" : 1689871805
}
key = "superSecretYouWillNeverGuess"
enc = jwt.encode(jwtPayload,key,algorithm='HS256',headers=jwtHeaders) print(enc)
```

F. **DATA SENDING**

1. **TESTING DATA SENDING (TESTING / CONNECTIVITY TEST):**
   i. API URL : https://sparing.kemenlh.go.id/api/testing
   ii. METHODE : POST
   iii. HEADER : {content-type: application/json}
   iv. BODY : SEND DATA (per 2 minutes)

2. **DATA SENDING EVERY 2 MINUTES:**
   i. API URL : https://sparing.kemenlh.go.id/api/send
   ii. METHODE : POST
   iii. HEADER : {content-type: application/json}
   iv. BODY : SEND DATA (per 2 minutes)

3. **HOURLY DATA SENDING (2-MINUTE INTERVAL):**
   i. API URL : https://sparing.kemenlh.go.id/api/send-hourly
   ii. METHODE : POST
   iii. HEADER : {content-type: application/json}
   iv. BODY : SEND DATA (Hourly)

4. **RETURN VALUES :**
   i. If VALID

```
{
    "status": true,
    "desc": null
}
```

   ii. If NOT VALID;

```
{
    "status": false,
    "desc": "……."
}
```